Universidad
Escuela de Ingeniería

LaTeX

Agregar Titulo del trabajo

Informe de Trabajo Final

Presenta:
Integrante

Ciudad - Pais
Fecha

Índice General

Resumen **v**

Abstract **v**

1. Introducción **1**

1.1. Objetivos 1

1.2. Objetivo general 1

1.3. Objetivo específicos 1

2. Marco teórico **3**

3. Diseño de la Solución **4**

4. Resultados y análisis **5**

Referencias Bibliográficas **6**

Apéndices **7**

A. Lista de Abreviaturas **7**

B. Ejemplos de notas **8**

B.1. Nota sin numerar 8

B.2. Nota numerada y referenciable 8

C. Ejemplos de código **9**

C.1. Barra de título: estándar 9

C.2. Barra de título: estéticas 10

C.3. Terminales oscuros 11

C.4. Sin barra de título 12

C.5. Formato anterior (`lstlisting`) 14

Índice de Tablas

Índice de Figuras

Resumen

Agregar resumen.

Palabras clave: agregar palabras.

Abstract

Resume.

Keywords: add keywords

1 INTRODUCCIÓN

1.1 Objetivos

1.2 Objetivo general

- Agregar objetivo general

1.3 Objetivo específicos

- Agregar objetivo especifico 1
- Agregar objetivo especifico 2
- Agregar objetivo especifico 3

Metodología de Verificación Universal (UVM) [1]

2 MARCO TEÓRICO

Introducción al marco teórico

3 DISEÑO DE LA SOLUCIÓN

4 RESULTADOS Y ANÁLISIS

Referencias Bibliográficas

[1] A. AUTORES, “Agregar titulo,” *AGREGAR REVISTA*, vol. AGREGAR VOLUMEN, no. AGREGAR NUMERO, p. AGREGAR PAGINAS, AGREGAR ANIO. [Online]. Available: AGREGARURL

Apéndices

A LISTA DE ABREVIATURAS

UVM Metodología de Verificación Universal

B EJEMPLOS DE NOTAS

Este apartado muestra una muestra de cada tipo de nota definido en `entornos.tex`. Ambas variantes comparten el mismo diseño (caja azul estilo IEEE); la diferencia es que la segunda se numera, se puede referenciar y aparece en el índice de notas.

B.1 Nota sin numerar

El entorno `notaieee` dibuja únicamente la caja y recibe el título como único argumento. Es adecuado para observaciones breves que no requieren referencia cruzada.

Nota IEEE sin numerar

Contenido de la nota. La caja admite texto corrido, expresiones en línea como $E = mc^2$ y listas:

- Primer punto de la nota.
- Segundo punto de la nota.

B.2 Nota numerada y referenciable

Para numerar la nota, la caja `notaieee` se coloca dentro del float `nota`, que aporta el `\caption` (numeración «Nota X.Y») y el `\label` para las referencias cruzadas.

Nota IEEE numerada

Esta nota se numera automáticamente y se registra en el índice de notas. Puede citarse en el texto con una referencia cruzada, como se hace a continuación.

Nota B.1. Ejemplo de nota numerada con `nota` y `notaieee`.

Referencia cruzada a la numeración: Nota B.1.

C EJEMPLOS DE CÓDIGO

Este apartado muestra una caja de cada tipo definida en `listings.tex`. Las cajas solo dibujan el marco; el `\caption`, el `\label` y la numeración «Código X.Y» los aporta el float `codigo` que las envuelve.

C.1 Barra de título: estándar

Barra oscura con un único argumento (el texto de la barra). Un entorno por lenguaje: `Ccode`, `SVcode` y `Bashcode`.

Ccode: barra oscura, sintaxis C

```c
#include <stdint.h>

/* Returns the sign bit of an IEEE 754 binary32 word */
static inline uint32_t f32_sign(uint32_t bits) {
    return bits >> 31;   // 1 = negative
}
```

Código C.1. Ejemplo con `Ccode`.

SVcode: barra oscura, sintaxis SystemVerilog

```systemverilog
module counter #(parameter int W = 8) (
  input  logic         clk, rst_n,
  output logic [W-1:0] count
);
  // Synchronous counter with active-low asynchronous reset
  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) count <= '0;
    else        count <= count + 1;
endmodule
```

Código C.2. Ejemplo con `SVcode`.

Bashcode: barra oscura, sintaxis bash

```bash
#!/bin/bash

# Build and run a single test
make clean
```

```bash
make all SEED=42
./run_sim.sh | tee sim.log   # keep a copy of the output
```

Código C.3. Ejemplo con `Bashcode`.

C.2 Barra de título: estéticas

Diseños alternativos con argumentos `[estilo]{barra}`. El estilo de sintaxis es opcional (por defecto C); aquí se usa el estilo por defecto en `Latcode` y se cambia en `Tabcode` y `Ieeecode`.

Latcode: franja lateral azul

```c
#include <stdint.h>

/* Returns the sign bit of an IEEE 754 binary32 word */
static inline uint32_t f32_sign(uint32_t bits) {
    return bits >> 31;   // 1 = negative
}
```

Código C.4. Ejemplo con `Latcode` (estilo por defecto `caja_c`).

Tabcode: pestaña oscura

```sv
module counter #(parameter int W = 8) (
  input  logic         clk, rst_n,
  output logic [W-1:0] count
);
  // Synchronous counter with active-low asynchronous reset
  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) count <= '0;
    else        count <= count + 1;
endmodule
```

Código C.5. Ejemplo con `Tabcode` y estilo `caja_sv`.

Ieeecode: barra azul IEEE

```bash
#!/bin/bash
# Build and run a single test
make clean
make all SEED=42
./run_sim.sh | tee sim.log   # keep a copy of the output
```

Código C.6. Ejemplo con `Ieeecode` y estilo `caja_bash`.

C.3 Terminales oscuros

Comparten los estilos de sintaxis y reglas de fondo oscuro (`caja_c_dark`, `caja_sv_dark` y `caja_bash_dark`) y se diferencian solo en los botones de la barra: círculos de macOS, tres círculos verdes y botón [+] de GNOME.

TermROGcode: terminal estilo macOS

```bash
cd build
make clean
make all SEED=42   # fixed seed for reproducibility
./run_sim.sh | tee sim.log
```

Código C.7. Ejemplo con `TermROGcode` y estilo `caja_bash_dark`.

TermGGGcode: terminal con círculos verdes

```c
#include <stdint.h>

/* Returns the sign bit of an IEEE 754 binary32 word */
static inline uint32_t f32_sign(uint32_t bits) {
    return bits >> 31;    // 1 = negative
}
```

Código C.8. Ejemplo con `TermGGGcode` (estilo por defecto `caja_c_dark`).

TermLinuxcode: terminal estilo GNOME

```sv
module counter #(parameter int W = 8) (
  input  logic         clk, rst_n,
  output logic [W-1:0] count
);
  // Synchronous counter with active-low asynchronous reset
  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) count <= '0;
    else        count <= count + 1;
endmodule
```

Código C.9. Ejemplo con `TermLinuxcode` y estilo `caja_sv_dark`.

C.4 Sin barra de título

Cajas sin barra superior. `MinCcode` es un marco fino sobre fondo blanco con estilo opcional (por defecto C); `MinCcapcode`, `MinSVcapcode` y `MinBashcapcode` no reciben argumentos y dibujan el marco sobre fondo gris.

```
$#include <stdint.h>

/* Returns the sign bit of an IEEE 754 binary32 word */
static inline uint32_t f32_sign(uint32_t bits) {
    return bits >> 31;   // 1 = negative
}
```

Código C.10. Ejemplo con `MinCcode` (estilo por defecto `caja_c`).

```
#include <stdint.h>
/* Returns the sign bit of an IEEE 754 binary32 word */
static inline uint32_t f32_sign(uint32_t bits) {
    return bits >> 31;   // 1 = negative
}
```

Código C.11. Ejemplo con `MinCcapcode`.

```
module counter #(parameter int W = 8) (
  input  logic         clk, rst_n,
  output logic [W-1:0] count
);
  // Synchronous counter with active-low asynchronous reset
  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) count <= '0;
    else        count <= count + 1;
endmodule
```

Código C.12. Ejemplo con `MinSVcapcode`.

```
#!/bin/bash
# Build and run a single test
make clean
make all SEED=42
./run_sim.sh | tee sim.log   # keep a copy of the output
```

Código C.13. Ejemplo con `MinBashcapcode`.

C.5 Formato anterior (`lstlisting`)

Formato independiente del float `codigo`: usa directamente `lstlisting` con los estilos `codigo` y `codigo_sv`, y la numeración nativa de `listings`.

```
#include <stdint.h>

/* Returns the sign bit of an IEEE 754 binary32 word */
static inline uint32_t f32_sign(uint32_t bits) {
    return bits >> 31;   // 1 = negative
}
```

Listing 1: Ejemplo con el formato anterior (`style=codigo`).

```
module counter #(parameter int W = 8) (
  input  logic         clk, rst_n,
  output logic [W-1:0] count
);
  // Synchronous counter with active-low asynchronous reset
  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) count <= '0;
    else        count <= count + 1;
endmodule
```

Listing 2: Ejemplo con el formato anterior (`style=codigo_sv`).